23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : D16

Team Members :

Roll No	Student Name
AM.SC.U4CSE25301	Amrithaleskshmy G
AM.SC.U4CSE25305	Arya S
AM.SC.U4CSE25316	Ganga T Viswakumar
AM.SC.U4CSE25317	Gloris S Mathew

Project Title : Personal Expense Tracker

Personal Expense Tracker is a database-backed application for recording and managing a user's personal income and expenses. A user creates an account and can maintain transaction records, organize transactions into categories, define monthly budgets, and view financial summaries and spending reports. The dashboard and reports are generated from the stored transaction and budget data.

The main entities are User, Category, Transaction, and Budget. Their roles are:

• User: stores the account and profile information of each application user.

• Category: stores transaction categories such as Food, Transport, Shopping, Utilities, Entertainment, Healthcare, Education, and Others. Categories are associated with a user.

• Transaction: stores each income or expense entry, including date, description, amount, transaction type, payment method, user, and category.

• Budget: stores a user's planned spending limit for a category for a particular month and year.

Users and business rules:

• A user can create many transactions, categories, and budgets.

• Every transaction belongs to exactly one user and one category.

• A category can be used by many transactions and can have multiple monthly budget records over time.

• A budget belongs to exactly one user and is associated with one category for a specified month and year.

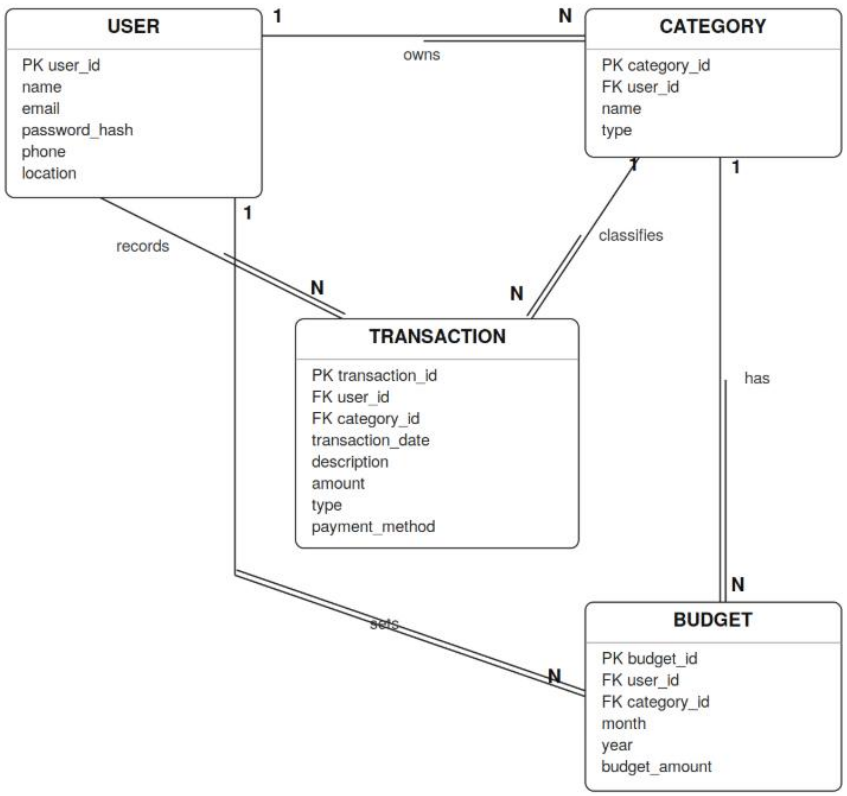
• Transaction type is restricted to Income or Expense.

• Amount must be positive; the transaction type determines whether the amount contributes to income or expense totals.

• A user's reports are derived from transactions and are not stored as a separate entity.

• For a user, a category should have at most one budget record for a particular month and year.

3.2. ER Diagram



3.3 Relational Schema

Relation	Relational Schema
User	USER(user_id PK, name, email UNIQUE, password_hash, phone, location)
Category	CATEGORY(category_id PK, user_id FK → USER.user_id, name, type)
Transaction	TRANSACTION(transaction_id PK, user_id FK → USER.user_id, category_id FK → CATEGORY.category_id, transaction_date, description, amount, type, payment_method)
Budget	BUDGET(budget_id PK, user_id FK → USER.user_id, category_id FK → CATEGORY.category_id, month, year, budget_amount, UNIQUE(user_id, category_id, month, year))

Relationship mapping:

• USER 1:N TRANSACTION is represented by TRANSACTION.user_id as a foreign key.

• USER 1:N CATEGORY is represented by CATEGORY.user_id as a foreign key.

• CATEGORY 1:N TRANSACTION is represented by TRANSACTION.category_id as a foreign key.

• USER 1:N BUDGET is represented by BUDGET.user_id as a foreign key.

• CATEGORY 1:N BUDGET is represented by BUDGET.category_id as a foreign key.

The Reports and Dashboard screens use aggregate queries over TRANSACTION and BUDGET rather than requiring separate stored relations.

3.4 Normalization and Functional dependency analysis

Relation	Functional Dependencies
USER	user_id → name, email, password_hash, phone, location; email → user_id, name, password_hash, phone, location (email is UNIQUE)
CATEGORY	category_id → user_id, name, type

TRANSACTION	transaction_id → user_id, category_id, transaction_date, description, amount, type, payment_method
BUDGET	(user_id, category_id, month, year) → budget_amount

Candidate-key justification using attribute closure:

• USER: {user_id}+ = {user_id, name, email, password_hash, phone, location}; therefore user_id is a candidate key. Because email is UNIQUE, {email}+ determines all USER attributes, so email is also a candidate key.

• CATEGORY: {category_id}+ = {category_id, user_id, name, type}; therefore category_id is a candidate key.

• TRANSACTION: {transaction_id}+ = all TRANSACTION attributes; therefore transaction_id is a candidate key.

• BUDGET: {user_id, category_id, month, year}+ = {user_id, category_id, month, year, budget_amount}; therefore this composite set is a candidate key. The surrogate budget_id is also a candidate key if it is defined as unique.

Normalization from 1NF to BCNF:

• 1NF: All relations contain atomic attributes and no repeating groups. Each row represents one user, category, transaction, or budget record.

• 2NF: USER, CATEGORY, and TRANSACTION have single-attribute candidate keys, so partial dependencies cannot occur. In BUDGET, the non-key attribute budget_amount depends on the whole composite candidate key (user_id, category_id, month, year), not on a proper subset.

• 3NF: There are no non-key attributes transitively determining other non-key attributes under the stated FDs. User, category, transaction, and budget facts are kept in separate relations.

• BCNF: For every non-trivial FD, the determinant is a superkey. In USER, user_id and the alternate key email are superkeys; in CATEGORY and TRANSACTION, their primary

keys are determinants; in BUDGET, the composite candidate key (user_id, category_id, month, year) is a determinant. Hence the proposed relations satisfy BCNF under the stated business rules.

Thus, the design avoids update, insertion, and deletion anomalies while keeping transaction, category, user, and budget information separated and consistently related through foreign keys.